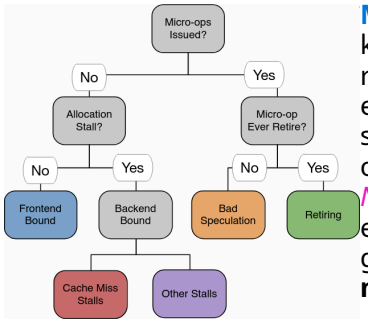
Introduction — *SWE Triangle*: general - maintainable - fast. *SMART*: Specific Measurable Acceptable Realisable Thorough. *Parameters*: system/workload, numeric/ nominal. *Utilisation*: percentage of a resource being used to perform a service. *Bottleneck*: resource with the *highest* utilisation. *Critical path*: **sequential** part of code in a parallel application that limits scaling through parallelisation. *Hot path*: path that **takes the most time**. An ideal system has no critical path. *Numerical model*: predict performance by interpolation between measured datapoints. *Analytical model*: stateful (Markov, stochastic), stateless. A formal characterisation of relationship between system parameters and performance metrics.

Tracing and Profiling — *Trace*: complete log of every state the system was in. *Perturbation*: system performance changing due to it being analysed. Reduce perturbation by sacrificing fidelity. *Time-based sampling*: set hardware recurrent timer and sample on interrupt. CPU cycles are innacurate, non-deterministic, noisy. *Quantisation error*: happens due to limited interval resolution. *Indirect tracing*: some trace events are dominated by others. Sample these instead: e.g. sample control-flow instructions to get an accurate trace. Indirect tracing has good fidelity and accuracy. *Profiling*: aggregation over a trace, focussing on specific metric. Profiling is easy to interpret and aggregation reduces perturbation. *Event sources*: software (library, compiler, kernel), hardware (performance counters), emulator (hybrid). *Instrumentation*: +no need for hardware support +flexible -overhead. *Static binary instrumentation*: binary is modified by injecting logging code. Reinstrumentation each time, cannot disable at runtime. *Dynamic binary instrumentation*: no recompilation, performed on running processes, works on JiT code. Example: LLVM XRay. *Software performance counters*: in kernel, e.g. packets sent, virtual memory operations. *Hardware performance counters*: special registers that count events. Fixed nº of registers. Often buggy, poorly documented, can have bad accuracy. *Pipeline*: Fetch → Decode → Execute → Memory → Write. *Control dependency stall*: e.g. branch misprediction causing pipeline flush. *Resource stall*: execution unit (e.g. ALU) not available. Long instructions can also cause pipeline bubbles. *Frontend-bound*: usually control-flow dependency pipeline flush or instruction cache miss. *Other stalls*: usually due to expensive ALU operations.



Modelling — we want to know performance without running the system, for cost estimation, provisioning systems and runtime optimisation/tuning. *Numerical model*: based on empirical datapoints gathered using **microbenchmarks**. *Microbenchmark*: small program designed to test specific portion of a system. Numerical models: easy, based on ground truth, generalise poorly, lots of data needed for high-dimensional parameter spaces, limited insight. *Analytical models*: require understanding. Needs a **characteristic equation**. Can use *model fitting*, e.g. linear regression. As stride varies, memory access time is given by:

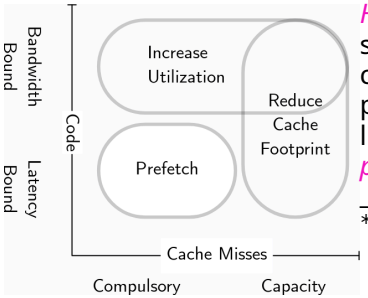
$$T_{\text{Mem}} = \sum_{i=o}^{3} l_i \cdot \min\left(1, \frac{s}{B_i}\right)$$

As array size changes, memory access is given by:

$$T_{\text{Mem}} = \begin{cases} l_0 & \text{size} < C_1 \\ l_0 + l_1 & \text{size} < C_2 \\ l_0 + l_1 + l_2 & \text{size} < C_3 \\ l_0 + l_1 + l_2 + l_3 & \text{otherwise} \end{cases}$$

Sequential memory access: reading u words then skipping $(R.w - u)$ words $R.n$ times. *Random access*: with repetitive access to elements. We have $R.w$, u, $\|R\|$ and an additional parameter r - the number of accesses. *Complex patterns*: define operators for sequential execution $\mathcal{P}_1 \oplus \mathcal{P}_2$ and concurrent execution $\mathcal{P}_1 \odot \mathcal{P}_2$. Example: $\text{s_trav}(R.w = 1, u = 1, R.n = 1024) \odot \text{rr_acc}(R.w = 1, u = 2, R.n = 64, r = 1024)$ *Branch predictors*: CPU stores *pattern history table* with four states: strongly not taken, weakly not taken, weakly taken, strongly taken. Branch misprediction rate is $P(\text{pred_taken}) \cdot P(\text{act_not_taken}) + P(\text{pred_not_taken}) \cdot P(\text{act_taken})$. These probabilities can be found using the stationary distribution of the Markov model.

Efficient code — *Balanced system*: one without a bottleneck where all resources are equally utilised. *Amdahl's Second Law*: A **balanced** computer system needs 1 MB of main memory and 1 Mb/s of I/O bandwidth per MIPS of CPU performance. Balance is not constant: datatypes are larger today, CPUs have many coprocessors, cache lines are larger. Balance is also a function of code: impacted by hardware optimisations. Perfect balance is an unachievable ideal. There is no balanced system, only sections of balanced code. We distinguish four types of code: *balanced*, *compute-bound*, *latency-bound*, *bandwidth-bound*. *Compute-bound*: related to CPU efficiency. Main metric: wallclock time. Stall cycles are a good indicator and are caused by *hazards*: control hazards, structural hazards (structure of CPU, i.e. lack of execution resources), data hazards (operands not available in time). These hazards can be mitigated in hardware by making more resources available. *Speculative execution*: branch + jump prediction. *Superscalar execution*: execute instructions in parallel by having more than one pipeline. *Out-of-order execution*: after decoding, CPU identifies instruction dependencies, allowing an instruction to overtake those it doesn't depend on. *SIMD*. *VLIW*: compiler-controlled superscalar. *Partial evaluation*: inlining, constant folding, metaprogramming, ... *If-conversion*: change branching to branch-free code. Turns control dependencies to data dependencies. Not worthwhile for predictable code. *Data hazard*: pipeline stall due to cache miss (capacity miss or compulsory miss).



Hardware prefetching: speculatively load next cache line, recognising patterns like adjacent cache lines, strides, ... *Software prefetching*: hint using `__builtin_prefetch(void *addr)`

Cache-line utilisation = data used by instructions ÷ data loaded into cache. *Thrashing*: can be fixed using *loop tiling*. *MESI*: Modified, Exclusive, Shared, Invalid. *False sharing*: another control-flow hazard.

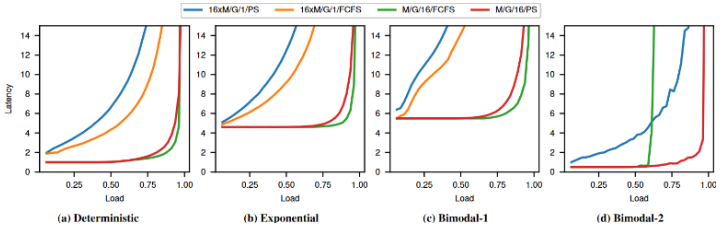
Multi-Core and Parallelism — *Amdahl's Law*:

$$\text{Speedup}(p, s) = 1 \div \left((1 - p) + \frac{p}{s}\right)$$

Synchronisation primitives: `std::mutex`, `std::unique_lock`, `std::lock_guard`, `std::counting_semaphore`, `std::binary_semaphore`, `std::shared_lock`, `std::condition_variable`, `std::barrier`. *False sharing*: parallel access to separate variables that live in the same cache line. Measure *cache line contention* through *HITM* count in `perf c2c`. *Naive threading*: when nº threads > nº cores: thread creation overhead, context switching/ scheduling overhead, cache locality degradation, lock

contention. Solution: *thread pool*. Either split jobs equally between threads, or use shared job queue to enable work stealing. *Streaming*: pipeline with concurrent stages (assembly line). Good locality for code & temporary data, bad locality for input/output data. *SEDA*: Staged Event-Driven Architecture. Modular stages w/ pools and input queue per stage. Bottlenecks are explicit (full queue). Observable load - can react by changing queue length, nº threads per stage. Very flexible tuning, ut stages across threads hurt performance (cache locality). *Multi-processing*: no shared memory, **expensive** communication. Can use explicit shared memory, sockets, pipes.

Tools, Algorithms and Patterns — *Blocking algorithms*: involve locking on critical sections, causing other threads to block. If a thread has a lock and gets de-scheduled, every other thread wanting that lock must wait. *Non-blocking*: suspension of one thread cannot cause suspension of other threads. Uses atomic RMW without locking. *Lock-free*: at least one thread is guaranteed to progress. *Wait-free*: every thread is guaranteed to progress. *ABA Problem*: wrong assumption that `head` being the same in the CAS also means `head->next` is the same. This can lead to `head` being assigned an out-of-date `head->next` that may have been freed. **System Interfaces and Performance —** *Syscall anatomy*: [user mode] store args in registers, syscall instr, [kernel mode] sanitize env, save state, execute handler, restore state, sys return instr, [user mode]. *Buffered IPC*: non-blocking, 2x data copies. *Unbuffered IPC*: blocking, 1x data copies. *Partial walk cache*: cache partial page table traversals. *TLB shootdown*: when a PTE is modified (e.g. `munmap` or page downgrade), the kernel sends IPIs to every core with that address space active to flush their TLB entries. The interrupts cause pipeline flushes, and the core that issued this needs to wait until all flushes have occurred. Solutions: avoid downgrades, asynchronous downgrade syscalls, hw acceleration. *Virtualisation*: reduce cost by sharing same physical machine, more secure than containers/processes. VM exceptions are **expensive**; long microcode to switch from guest/host, saves/restores several registers. *Virtual memory translation*: let hypervisor manage PTs (*shadow paging*); expensive vm-exits to manage guest PTs. Or use guest and host PWCs. *MMIO*: Use PCIe BARs (base addr registers), circular buffer in host memory and BARs configure queue changes. *Device driver models*: interrupt-driven, polling (using MMIO reads), hybrid (dynamically switch to polling if latency is low). *Device virtualisation models*: trap & emulate (**very expensive**), para-virtualised (map shared memory between VM & hypervisor, trapped MMIO access as doorbell signal), passthrough (map device BARs into VM & configure IOMMU, devices need one set of BARs per VM). *Non-blocking interfaces* avoid context switch & data copy. Can be asynchronous or event-based. *Asynchronous interfaces*: kernel signals end of operation. *Event-based interfaces*: express interest on ops on FDs. Poll kernel for available ops in event loop, e.g. `epoll` tells you when a socket is ready to read from. *Direct device assignment*: bypass kernel with user-mode MMIO, e.g. DPDK. **System Programming Models —** *One-thread per task*: main thread spawns worker thread for each new connection. Thread creation is expensive (time + memory), spends lots of time context switching. *Worker pools*: avoids cost of on0demand thread creation, no clients processed when workers are blocked, still has context switch overhead if nº threads > nº CPUs. *Event-based*: pin one thread per core, use non-blocking IO, keep state machine to track each concurrent context. no thread scheduling, minimal memory usage, complicated to program. *Memcached*: in-memory key/value store. calls to read/write incur copies, bad locality at high load (app. & network stack data & locks go across cores). *IX*: NIC uses ECMP to steer packets to cores. VM with NIC passthrough, zero copy, syscall batching, end-to-end processing maximises cache locality. **Scale-Out Systems —** *Energy proportionality*: ideally energy consumption $\propto$ utilization. Reality: substantial minimum cost + energy at low loads. Best value when hardware is off, or at max utilisation. Increasing utilisation does reduce cost but can easily violate SLO. *Elasticity*: ability to adapt to local changes, e.g. add/remove CPUs according to load. *Scale-out*: **handle failures**. *Microservices*: one service for each task, increase capacity by increasing instances. Complex management, distribution, VMs have expensive boot time. Can isolate tenants with VMs and isolate services using containers. *Serverless*: tenant only declares what to run, on certain events. Operator load-balances & auto-scales. *The standing problem*: starting and stopping serverless instances is expensive: container creation (setting up namespaces), service boot (runtime initialisation). Solution: **cache and prefetch**. *Function-as-a-service* (FaaS): tenant provides business logic and **trigger** stateless function. Tenants use separate VMs (security), one container + runtime per function. Limited VM/container/runtime configs, easier to predict, prefetch and reuse. **Communication Mechanisms —** HTTP REST + JSON most common solution. *RPCs*: remote procedure calls. Server API defined as function calls (usually asynchronous). Frameworks: gRPC, protobuf, ... Binary-based: more efficient to transfer & compress. *The standing problem*: RESTful code & RPC stubs transfer lots of data buffers. Lots of cycles spent on data transfer alone: allocate/free & send/receive buffers. *RDMA*: Remote DMA. Two types of primitives: stream (send/receive) & memory (read/write). Two modes: reliable & unreliable. moves network stack into NIC; less CPU load, hard to program. OS needs to send virtual → physical memory translations to NIC. Paging application memory out requires telling NIC to invalidate its translations (expensive). *Tail latency*: key performance goal. Caused by transient overheads (lock contention, TLB misses, etc.) and app-level differences (e.g. get vs set). Utilisation ↓ tail latency ↓, utilisation ↑ costs ↓. *Load balancing*: distributing requests over resources. Can distribute over *time*, *data*. Scale: *cross-core*, *cross-machine*. **Queueing Theory —** three main parameters define **service latency**: arrival distribution (Poisson), arrival assignment, service time distribution (ideally constant, realistically exponential). *Parameters*: single-queue vs multi-queue, FCFS vs processor sharing. *Goals*: low tail latency, *work conservation* (core never idle if there's work to do).



IX: lower tail latency, 32xM/G/1/FCFS system. Perfect on constant/exponential service times. Designed to "narrow" exponential dist. HOL on bimodal service dist. *ZygOS*: scale-up, almost single-queue, supports large service dispersion, work-conserving, avoids HOL blocking. *R2P2*: scale-out, towards single queue for all service instances. Global queue when establishing connection, regular client/server connection after, doesn't rebalance long-lived connections, reduce tail latency due to overloaded nodes. *Load proportionality*: PEGASUS; monitor app metrics, use DVVFS to change core speed (power). *Scale-out* has large waste when nodes are underutilised: exploit elasticity. Microservices make it simple, serverless & FaaS make it efficient, FaaS make it adaptable.